Calculadora en Python

Práctica 2 · Módulo 2 · Sesión 17, Python para Análisis de Datos · Luis Saúl González Guerra · Noviembre 2026

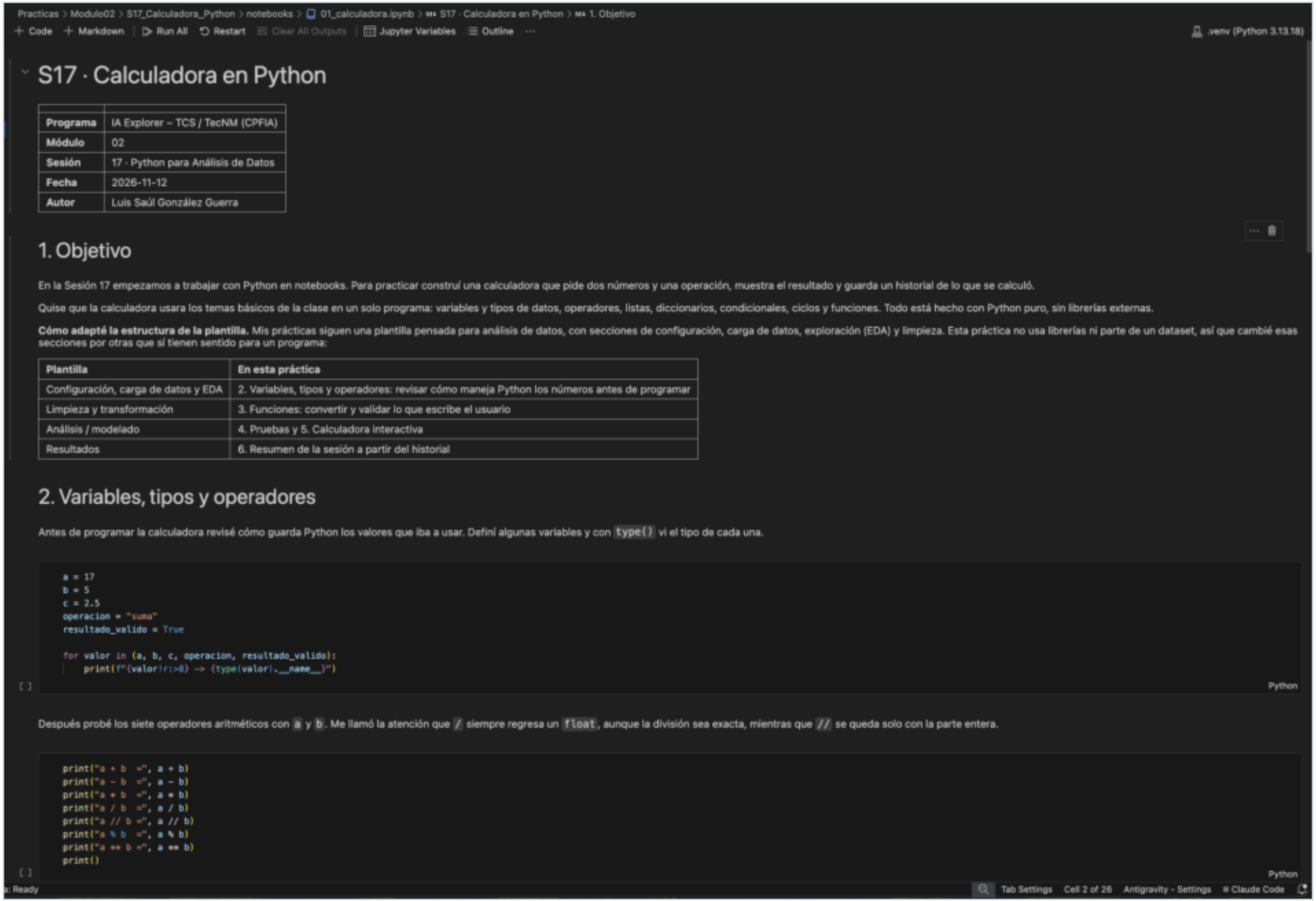
Temas de la sesión y dónde los usé en la calculadora

Tema	Dónde lo usé
Variables y tipos de datos	int, float, str y bool; type() para revisarlos
Operadores	+ - * / // % ** en las siete operaciones; comparación y lógicos para validar
Listas	Casos de prueba y el historial de operaciones
Diccionarios	Cada símbolo asociado a su función; cada operación del historial; conteo por operación
Condicionales y ciclos	if para validar, for para las pruebas, while para la calculadora
Funciones	Una función por operación, más convertir_numero y calcular
Errores comunes	TypeError, ValueError y ZeroDivisionError manejados con try / except

Herramientas: Python 3.13 y notebooks de Jupyter en VS Code, sin librerías externas. El notebook completo está al final del reporte.

ENTORNO

Notebook abierto en VS Code con el entorno .venv del repositorio



PRUEBAS

Pruebas de las siete operaciones

4. Pruebas

Antes de usar la calculadora quise comprobar que cada operación diera lo que yo esperaba. Armé una lista de casos; cada caso es una tupla con los dos números, el símbolo y el resultado que calculé a mano. Con un `for` recorrí la lista, comparé lo obtenido con lo esperado y conté cuántas pruebas pasaron. En las divisiones entre cero lo que espero es el error.

```python
casos = [
    (8, "+", 5, 13),
    (8, "-", 5, 3),
    (8, "*", 5, 40),
    (8, "/", 5, 1.6),
    (17, "//", 5, 3),
    (17, "%", 5, 2),
    (2, "**", 10, 1024),
    (-7, "//", 2, -4),
    (0.1, "+", 0.2, 0.3),
    (9, "/", 0, "error"),
    (9, "%", 0, "error"),
]

aprobadas = 0
for x, simbolo, y, esperado in casos:
    try:
        obtenido = calcular(x, simbolo, y)
    except ZeroDivisionError:
        obtenido = "error"
    paso = obtenido == esperado
    if paso:
        aprobadas += 1
    print(f"{'OK   ' if paso else 'FALLA'}  {x} {simbolo} {y} = {obtenido}")

print(f"\n{aprobadas} de {len(casos)} pruebas correctas")
```

[9] ✓ 0.0s Python

```
OK     8 + 5 = 13
OK     8 - 5 = 3
OK     8 * 5 = 40
OK     8 / 5 = 1.6
OK     17 // 5 = 3
OK     17 % 5 = 2
OK     2 ** 10 = 1024
OK     -7 // 2 = -4
OK     0.1 + 0.2 = 0.3
OK     9 / 0 = error
OK     9 % 0 = error

11 de 11 pruebas correctas
```

CALCULADORA INTERACTIVA

Sesión de uso de la calculadora

5. Calculadora interactiva

Aquí junté todo dentro de un ciclo `while`. La calculadora pide el primer número, la operación y el segundo número, muestra el resultado y vuelve a empezar hasta que escribo `salir`. Si escribo algo que no es número, una operación que no existe o intento dividir entre cero, muestra el error y sigue funcionando en lugar de detenerse.

Cada operación se guarda en la lista `historial` como un diccionario, incluso las que dieron error, para poder revisarlas al final.

```python
def calculadora():
    historial = []
    print("Operaciones:", "  ".join(OPERACIONES))
    print("Escribe 'salir' en el primer número para terminar.\n")

    while True:
        texto_x = input("Primer número: ").strip()
        if texto_x.lower() == "salir":
            break
        simbolo = input("Operación: ").strip()
        texto_y = input("Segundo número: ").strip()

        registro = {
            "numero_1": texto_x,
            "operacion": simbolo,
            "numero_2": texto_y,
            "nombre": OPERACIONES.get(simbolo, {}).get("nombre", "No válida"),
            "resultado": None,
            "error": None,
        }
        try:
            x = convertir_numero(texto_x)
            y = convertir_numero(texto_y)
            registro["resultado"] = calcular(x, simbolo, y)
            print(f"  {x} {simbolo} {y} = {registro['resultado']}\n")
        except (ValueError, ZeroDivisionError, OverflowError) as error:
            registro["error"] = type(error).__name__
            print(f"  Error: {error}\n")
        historial.append(registro)

    print(f"Fin. Operaciones registradas: {len(historial)}")
    return historial


historial = calculadora()
```

salir

Primer número: (Press 'Enter' to confirm or 'Escape' to cancel)

3m 4.3s — Python

```
Operaciones: +  -  *  /  //  %  **
Escribe 'salir' en el primer número para terminar.

  273 + 18273 = 18546

  Error: '22**' no es un número

  Error: '' no es un número

  5 ** 5 = 3125

  2 ** 2 = 4

  Error: no se puede dividir entre cero

  273 / 7 = 39.0

  Error: 'treinta' no es un número

  Error: 'd' no es un número

  Error: la operación '^' no existe
```

RESUMEN

Resumen de la sesión a partir del historial

6. Resumen de la sesión

Al terminar recorrí el historial con un `for` para mostrar cada operación con su resultado o su error. En el mismo ciclo conté las operaciones correctas y, con un diccionario, cuántas veces usé cada operación: si el nombre todavía no estaba como clave empiezo en 0 con `get()` y le sumo 1.

```python
print(f"{'#':>2}  {'Operación':<22} Resultado")
for numero, registro in enumerate(historial, start=1):
    texto = f"{registro['numero_1']} {registro['operacion']} {registro['numero_2']}"
    if registro["error"] is None:
        salida = registro["resultado"]
    else:
        salida = f"Error ({registro['error']})"
    print(f"{numero:>2}  {texto:<22} {salida}")

conteo = {}
correctas = 0
for registro in historial:
    nombre = registro["nombre"]
    conteo[nombre] = conteo.get(nombre, 0) + 1
    if registro["error"] is None:
        correctas += 1
errores = len(historial) - correctas

print(f"\nTotal: {len(histor Literal['\nVeces que usé cada operación:'] errores} con error.")
print("\nVeces que usé cada operación:")
for nombre, veces in sorted(conteo.items(), key=lambda par: par[1], reverse=True):
    print(f"  {nombre:<16} {veces}")
```

[11] ✓ 0.0s Python

```
 #  Operación             Resultado
 1  273 + 18273           18546
 2  22** **               Error (ValueError)
 3  22 **                 Error (ValueError)
 4  5 ** 5                3125
 5  2 ** 2                4
 6  100 / 0               Error (ZeroDivisionError)
 7  273 / 7               39.0
 8  treinta + 1           Error (ValueError)
 9  d ^ 5                 Error (ValueError)
10  3 ^ 6                 Error (ValueError)

Total: 10 operaciones, 4 correctas y 6 con error.

Veces que usé cada operación:
  Potencia         4
  Suma             2
  División         2
  No válida        2
```

S17 · Calculadora en Python

Programa	IA Explorer – TCS / TecNM (CPFIA)
Módulo	02
Sesión	17 · Python para Análisis de Datos
Fecha	2026-11-12
Autor	Luis Saúl González Guerra

1. Objetivo

En la Sesión 17 empezamos a trabajar con Python en notebooks. Para practicar construí una calculadora que pide dos números y una operación, muestra el resultado y guarda un historial de lo que se calculó.

Quise que la calculadora usara los temas básicos de la clase en un solo programa: variables y tipos de datos, operadores, listas, diccionarios, condicionales, ciclos y funciones. Todo está hecho con Python puro, sin librerías externas.

Cómo adapté la estructura de la plantilla. Mis prácticas siguen una plantilla pensada para análisis de datos, con secciones de configuración, carga de datos, exploración (EDA) y limpieza. Esta práctica no usa librerías ni parte de un dataset, así que cambié esas secciones por otras que sí tienen sentido para un programa:

Plantilla	En esta práctica
Configuración, carga de datos y EDA	2. Variables, tipos y operadores: revisar cómo maneja Python los números antes de programar
Limpieza y transformación	3. Funciones: convertir y validar lo que escribe el usuario
Análisis / modelado	4. Pruebas y 5. Calculadora interactiva
Resultados	6. Resumen de la sesión a partir del historial

2. Variables, tipos y operadores

Antes de programar la calculadora revisé cómo guarda Python los valores que iba a usar. Definí algunas variables y con `type()` vi el tipo de cada una.

In [1]:
```
a = 17
b = 5
c = 2.5
operacion = "suma"
resultado_valido = True

for valor in (a, b, c, operacion, resultado_valido):
    print(f"{valor!r:>8} -> {type(valor).__name__}")
```

```
    17 -> int
     5 -> int
   2.5 -> float
 'suma' -> str
  True -> bool
```

Después probé los siete operadores aritméticos con `a` y `b`. Me llamó la atención que `/` siempre regresa un `float`, aunque la división sea exacta, mientras que `//` se queda solo con la parte entera.

In [2]:
```
print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)
print("a // b =", a // b)
print("a % b  =", a % b)
print("a ** b =", a ** b)
print()
print("10 / 2 =", 10 / 2, "->", type(10 / 2).__name__)
```

```
a + b  = 22
a - b  = 12
a * b  = 85
a / b  = 3.4
a // b = 3
a % b  = 2
a ** b = 1419857

10 / 2 = 5.0 -> float
```

Los operadores de comparación y los lógicos (`and`, `or`, `not`) los iba a necesitar para validar. Por ejemplo, una división solo se puede hacer si el divisor es distinto de cero.

In [3]:
```
divisor = 0
print("¿Puedo dividir entre", divisor, "?", divisor != 0)

x = 8
print("¿x es par y mayor que 5?", x % 2 == 0 and x > 5)
print("¿x es negativo o cero?", x < 0 or x == 0)
print("¿x es distinto de 8?", not x == 8)
```

```
¿Puedo dividir entre 0 ? False
¿x es par y mayor que 5? True
¿x es negativo o cero? False
¿x es distinto de 8? False
```

Errores que aparecieron

En clase vimos los errores más comunes. Al probar me encontré con un `TypeError` al sumar un texto con un número, que es justo lo que pasa con `input()`: todo lo que escribe el usuario llega como texto. También revisé qué pasa al dividir entre cero y al convertir una palabra a número. Usé `try` / `except` para que el notebook no se detuviera y poder ver el mensaje de cada error.

In [4]:
```
entrada = "12"  # así llega un número desde input()

try:
    print(entrada + 3)
except TypeError as error:
    print("TypeError:", error)
```

```
print("Convirtiendo primero con float():", float(entrada) + 3)

try:
    print(10 / 0)
except ZeroDivisionError as error:
    print("ZeroDivisionError:", error)

try:
    float("doce")
except ValueError as error:
    print("ValueError:", error)
```

```
TypeError: can only concatenate str (not "int") to str
Convirtiendo primero con float(): 15.0
ZeroDivisionError: division by zero
ValueError: could not convert string to float: 'doce'
```

3. Funciones de la calculadora

Escribí una función por operación. Cada una recibe dos números y regresa el resultado con `return`. La división, la división entera y el módulo primero revisan que el divisor no sea cero; si lo es, lanzan un `ZeroDivisionError` con un mensaje en español.

In [5]:
```python
def sumar(x, y):
    return x + y


def restar(x, y):
    return x - y


def multiplicar(x, y):
    return x * y


def revisar_divisor(y):
    if y == 0:
        raise ZeroDivisionError("no se puede dividir entre cero")


def dividir(x, y):
    revisar_divisor(y)
    return x / y


def division_entera(x, y):
    revisar_divisor(y)
    return x // y


def modulo(x, y):
    revisar_divisor(y)
    return x % y


def potencia(x, y):
    return x ** y
```

Para no llenar la calculadora de `if` / `elif` con cada símbolo, guardé las operaciones en un diccionario. La clave es el símbolo que se escribe y el valor es otro diccionario con el nombre de la operación y la función que la resuelve.

In [6]:
```python
OPERACIONES = {
    "+": {"nombre": "Suma", "funcion": sumar},
    "-": {"nombre": "Resta", "funcion": restar},
    "*": {"nombre": "Multiplicación", "funcion": multiplicar},
    "/": {"nombre": "División", "funcion": dividir},
    "//": {"nombre": "División entera", "funcion": division_entera},
    "%": {"nombre": "Módulo", "funcion": modulo},
    "**": {"nombre": "Potencia", "funcion": potencia},
}

for simbolo, datos in OPERACIONES.items():
    print(f"{simbolo:>3}  {datos['nombre']}")
```

```
 +   Suma
 -   Resta
 *   Multiplicación
 /   División
//   División entera
 %   Módulo
**   Potencia
```

Luego hice dos funciones más:

- `convertir_numero` pasa a número el texto que llega de `input()`. Acepta enteros y decimales, y si el número no tiene parte decimal lo regresa como `int` para que `5` no aparezca como `5.0`. Si el texto no es un número, lanza un `ValueError` con un mensaje claro.
- `calcular` busca el símbolo en el diccionario y llama a la función que le corresponde. Los resultados con decimales los redondeo a seis posiciones (más abajo explico por qué).

In [7]:
```python
def convertir_numero(texto):
    try:
        numero = float(texto)
    except ValueError:
        raise ValueError(f"'{texto}' no es un número") from None
    if numero.is_integer():
        return int(numero)
    return numero


def calcular(x, simbolo, y):
    if simbolo not in OPERACIONES:
        raise ValueError(f"la operación '{simbolo}' no existe")
    funcion = OPERACIONES[simbolo]["funcion"]
    resultado = funcion(x, y)
    if isinstance(resultado, float):
        resultado = round(resultado, 6)
    return resultado


print(calcular(convertir_numero("17"), "/", convertir_numero("4")))
print(calcular(convertir_numero("2.5"), "*", convertir_numero("4")))
```

```
4.25
10.0
```

4. Pruebas

Antes de usar la calculadora quise comprobar que cada operación diera lo que yo esperaba. Armé una lista de casos; cada caso es una tupla con los dos números, el símbolo y el resultado que calculé a mano. Con un `for` recorrí la lista, comparé lo obtenido con lo esperado y conté cuántas pruebas pasaron. En las divisiones entre cero lo que espero es el error.

In [8]:
```python
casos = [
    (8, "+", 5, 13),
    (8, "-", 5, 3),
```

```
    (8, "*", 5, 40),
    (8, "/", 5, 1.6),
    (17, "//", 5, 3),
    (17, "%", 5, 2),
    (2, "**", 10, 1024),
    (-7, "//", 2, -4),
    (0.1, "+", 0.2, 0.3),
    (9, "/", 0, "error"),
    (9, "%", 0, "error"),
]

aprobadas = 0
for x, simbolo, y, esperado in casos:
    try:
        obtenido = calcular(x, simbolo, y)
    except ZeroDivisionError:
        obtenido = "error"
    paso = obtenido == esperado
    if paso:
        aprobadas += 1
    print(f"{'OK   ' if paso else 'FALLA'}  {x} {simbolo} {y} =
{obtenido}")

print(f"\n{aprobadas} de {len(casos)} pruebas correctas")
```

```
OK     8 + 5 = 13
OK     8 - 5 = 3
OK     8 * 5 = 40
OK     8 / 5 = 1.6
OK     17 // 5 = 3
OK     17 % 5 = 2
OK     2 ** 10 = 1024
OK     -7 // 2 = -4
OK     0.1 + 0.2 = 0.3
OK     9 / 0 = error
OK     9 % 0 = error

11 de 11 pruebas correctas
```

Dos de los casos los agregué por curiosidad:

- `-7 // 2` da `-4` y no `-3`, porque `//` redondea hacia abajo, no hacia el cero.
- `0.1 + 0.2` en Python da `0.30000000000000004`, por cómo se guardan los decimales en binario. Por eso en `calcular` redondeo los resultados a seis decimales; sin ese redondeo esta prueba falla.

In [9]:
```
print(-7 // 2)
print(0.1 + 0.2)
```

```
-4
0.30000000000000004
```

5. Calculadora interactiva

Aquí junté todo dentro de un ciclo `while`. La calculadora pide el primer número, la operación y el segundo número, muestra el resultado y vuelve a empezar hasta que

escribo `salir`. Si escribo algo que no es número, una operación que no existe o intento dividir entre cero, muestra el error y sigue funcionando en lugar de detenerse.

Cada operación se guarda en la lista `historial` como un diccionario, incluso las que dieron error, para poder revisarlas al final.

In [10]:
```
def calculadora():
    historial = []
    print("Operaciones:", "  ".join(OPERACIONES))
    print("Escribe 'salir' en el primer número para terminar.\n")

    while True:
        texto_x = input("Primer número: ").strip()
        if texto_x.lower() == "salir":
            break
        simbolo = input("Operación: ").strip()
        texto_y = input("Segundo número: ").strip()

        registro = {
            "numero_1": texto_x,
            "operacion": simbolo,
            "numero_2": texto_y,
            "nombre": OPERACIONES.get(simbolo, {}).get("nombre", "No
válida"),
            "resultado": None,
            "error": None,
        }
        try:
            x = convertir_numero(texto_x)
            y = convertir_numero(texto_y)
            registro["resultado"] = calcular(x, simbolo, y)
            print(f"  {x} {simbolo} {y} = {registro['resultado']}\n")
        except (ValueError, ZeroDivisionError, OverflowError) as error:
            registro["error"] = type(error).__name__
            print(f"  Error: {error}\n")
        historial.append(registro)

    print(f"Fin. Operaciones registradas: {len(historial)}")
    return historial


historial = calculadora()
```

```
Operaciones: +  -  *  /  //  %  **
Escribe 'salir' en el primer número para terminar.

  273 + 18273 = 18546

  Error: '22**' no es un número

  Error: '' no es un número

  5 ** 5 = 3125

  2 ** 2 = 4

  Error: no se puede dividir entre cero

  273 / 7 = 39.0

  Error: 'treinta' no es un número

  Error: 'd' no es un número

  Error: la operación '^' no existe

Fin. Operaciones registradas: 10
```

6. Resumen de la sesión

Al terminar recorrí el historial con un `for` para mostrar cada operación con su resultado o su error. En el mismo ciclo conté las operaciones correctas y, con un diccionario, cuántas veces usé cada operación: si el nombre todavía no estaba como clave empiezo en 0 con `get()` y le sumo 1.

In [11]:
```python
print(f"{'#':>2}  {'Operación':<22} Resultado")
for numero, registro in enumerate(historial, start=1):
    texto = f"{registro['numero_1']} {registro['operacion']}
{registro['numero_2']}"
    if registro["error"] is None:
        salida = registro["resultado"]
    else:
        salida = f"Error ({registro['error']})"
    print(f"{numero:>2}  {texto:<22} {salida}")

conteo = {}
correctas = 0
for registro in historial:
    nombre = registro["nombre"]
    conteo[nombre] = conteo.get(nombre, 0) + 1
    if registro["error"] is None:
        correctas += 1
errores = len(historial) - correctas

print(f"\nTotal: {len(historial)} operaciones, {correctas} correctas y
{errores} con error.")
print("\nVeces que usé cada operación:")
for nombre, veces in sorted(conteo.items(), key=lambda par: par[1],
reverse=True):
    print(f"  {nombre:<16} {veces}")
```

```
 #  Operación             Resultado
 1  273 + 18273           18546
 2  22** **               Error (ValueError)
 3  22 **                 Error (ValueError)
 4  5 ** 5                3125
 5  2 ** 2                4
 6  100 / 0               Error (ZeroDivisionError)
 7  273 / 7               39.0
 8  treinta + 1           Error (ValueError)
 9  d ^ 5                 Error (ValueError)
10  3 ^ 6                 Error (ValueError)

Total: 10 operaciones, 4 correctas y 6 con error.

Veces que usé cada operación:
  Potencia         4
  Suma             2
  División         2
  No válida        2
```

7. Conclusiones

- Lo más importante que noté es que `input()` siempre regresa texto. Antes de calcular hay que convertirlo a número, y es justo ahí donde aparecen la mayoría de los errores.
- Guardar las funciones dentro de un diccionario me ahorró una cadena larga de `if` / `elif`. Para agregar una operación nueva solo tengo que escribir su función y agregar una línea al diccionario.
- Separar el código en funciones pequeñas me permitió probar cada operación por separado antes de armar la calculadora completa.
- Con `try` / `except` la calculadora sigue funcionando aunque se escriba algo mal, en lugar de detenerse con un error.
- Descubrí detalles de Python que no esperaba: `/` siempre da decimal, `//` redondea hacia abajo con negativos y `0.1 + 0.2` no da exactamente `0.3`.
- Guardar cada operación como un diccionario dentro de una lista me permitió recorrer el historial al final y contar las operaciones con otro diccionario.

Como siguiente paso me gustaría agregar raíz cuadrada y porcentaje, y que la calculadora pueda usar el resultado anterior como primer número.

8. Referencias

- TCS Empowers / TecNM CPFIA. *Sesión 17: Python para Análisis de Datos* (Basic Notebook con Python).
- Python Software Foundation. Tutorial de Python.
- Python Software Foundation. Errores y excepciones.